

CE 488/5588: Applied AI

Topic 17: Agentic AI, Human-in-the-Loop
Engineering, and Accountability

Lecture Roadmap

- 1 From Topic 16 to Topic 17
- 2 What Makes a System Agentic?
- 3 Core Components of an Agentic Workflow
- 4 Skills, Tool Interfaces, and Orchestration
- 5 Automated Research Assistants
- 6 Human-in-the-Loop Engineering
- 7 Failure Modes of Agentic AI
- 8 Trust, Accountability, and Governance
- 9 Representative Agentic Systems in Practice
- 10 Conclusion and Transition

From Structured Workflows to Agentic Systems

Topic 16 introduced:

- prompt workflows,
- retrieval-augmented generation,
- knowledge-graph support,
- ReAct-style loops,
- direct engineering applications.

Topic 17 asks what changes when those pieces become persistent, tool-using, and multi-step.

The Significance of Topic 17

Many engineering tasks are not single-turn tasks.
A useful workflow may require:

- reading multiple reports,
- extracting assumptions,
- checking standards,
- comparing versions,
- writing code,
- escalating uncertainty to a human reviewer.

Main Shift

The useful unit of work is often a workflow, not a single answer.

Scope of Topic 17

This chapter focuses on:

- what makes a workflow agentic,
- planning, tools, memory, and recovery,
- skills and orchestration,
- automated research assistants,
- human-in-the-loop engineering,
- trust, accountability, and governance.

It does not assume that more autonomy is always better.

Learning Objectives

After this chapter, students should be able to:

- explain what makes an AI workflow agentic,
- distinguish planning, tool use, memory, and execution,
- describe skills, tool interfaces, and orchestration,
- explain why HITL design remains central,
- identify major agentic-AI failure modes,
- explain why accountability grows as autonomy grows.

A Working Definition of Agentic AI

The term **agentic AI** is often used loosely.

Working Definition

A system becomes agentic when it can pursue a goal over multiple steps by combining planning, action, intermediate state, and adaptation to observations.

That is more than one-shot prompting, but not necessarily full autonomy.

What the Definition Emphasizes

Four ideas matter:

- **goal persistence** — complete a task, not just answer once,
- **action selection** — choose among tools or operations,
- **state** — remember what has already happened,
- **adaptation** — change later actions based on new observations.

Unified Architecture of Agentic AI

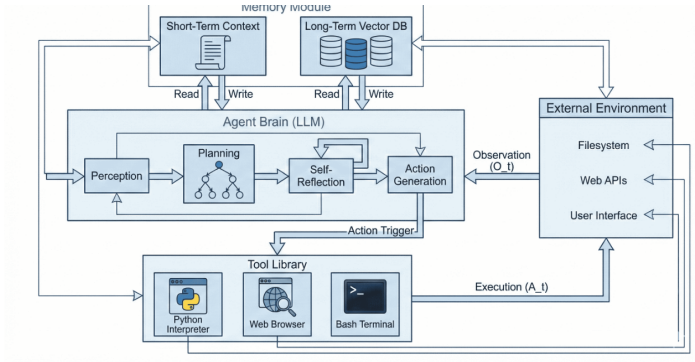


Figure: Unified architecture of agentic AI.

Source: Arunkumar, Gangadharan, and Buyya (2026): Agentic Artificial Intelligence

Significance of the Agentic Architecture

This architecture figure makes the progression beyond Topic 16 explicit.

- The model is no longer isolated.
- Memory and tools are part of the system architecture.
- The environment matters because observations affect later actions.
- Multi-step behavior becomes a workflow property, not only a prompt property.

Agentic Does Not Mean Fully Autonomous

In practice, there is a spectrum:

- tool-assisted chat,
- bounded workflow agents,
- semi-autonomous agents,
- longer-horizon frontier systems.

Engineering View

The right design is usually not maximum autonomy. It is the minimum autonomy that creates useful leverage while preserving control.

Four Core Components

Most agentic workflows can be understood as combinations of:

- planning,
- tool use,
- memory/state,
- execution with recovery.

The exact implementation varies, but these components recur across most practical systems.

Planning

Planning decomposes a large task into intermediate steps. It matters because many AI failures are actually failures of decomposition.

Planning Lesson

If a large engineering task is treated as one undifferentiated prompt, the output tends to be fluent but weak. Breaking it into retrieval, comparison, verification, and synthesis improves reliability.

Tool Use and Memory

Tool use lets the system interact with external capabilities such as:

- document retrieval,
- code execution,
- spreadsheet parsing,
- file inspection,
- web search.

Memory keeps track of retrieved evidence, completed subtasks, unresolved questions, and prior failures.

Execution and Recovery

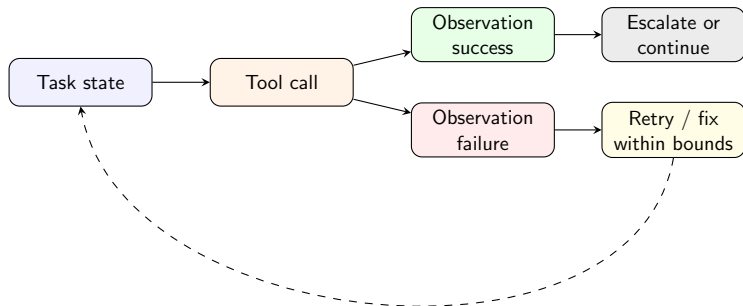


Figure: Execution and recovery in an agentic workflow.

Significance of Recovery Logic

A robust workflow does not continue blindly after failure.
It should:

- inspect the failure,
- attempt a bounded fix,
- escalate when uncertainty becomes too high.

In engineering, silent failure may be worse than explicit failure.

Three Reusable Abstractions

As workflows become more capable, they are often organized around:

- **skills,**
- **tool interfaces,**
- **orchestration layers.**

These abstractions reduce improvisation and improve repeatability.

Skills as Reusable Task Patterns

A skill is a reusable pattern for a class of work.

Examples in engineering settings:

- manuscript review,
- note-to-slide conversion,
- codebase exploration,
- report validation,
- handoff generation.

Skills make systems more consistent by reducing ad hoc workflow invention.

Tool Interfaces and Orchestration Layers

Tool interfaces define how external capabilities are called.
Orchestration layers decide:

- which skill to invoke,
- which tool to call,
- whether to continue, retry, or escalate,
- how to preserve intermediate state,
- how to route work among specialized modules.

System View

Frontier AI systems increasingly look like software architectures rather than just models.

Engineering Interpretation

Suppose an organization wants an AI assistant for bridge-inspection documentation.

One skill may:

- summarize field notes,
- compare them against prior reports,
- search standards for relevant clauses.

The orchestration layer determines when to call each skill and when a human reviewer must intervene.

Automated Research Assistants

A research-style agent is not a system that “knows everything.”

It is a system that can:

- search multiple sources,
- compare evidence,
- track unresolved questions,
- produce a structured synthesis.

This is one of the clearest real-world frontier applications of agentic AI.

Verification-Driven Multi-Agent Orchestration

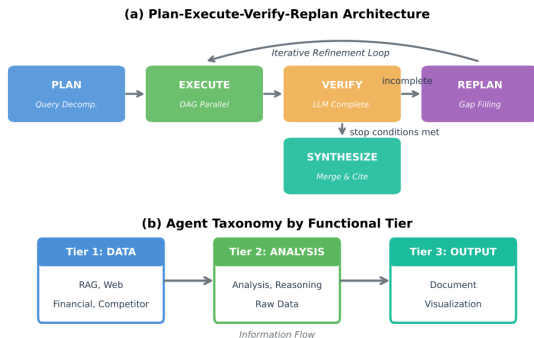


Figure: Verification-driven multi-agent orchestration.

Source: Zhang et al. (2026): Verified Multi-Agent Orchestration (VMAO)

Significance of Verification-Driven Research Workflows

The VMAO diagram is pedagogically useful because it makes the verifier explicit.

- Planning is only the first step.
- Execution may happen in parallel.
- Verification checks completeness.
- Replanning occurs when gaps remain.

That is stronger than a generic loop because it makes quality control visible.

Significance for Engineering Practice

Engineering work contains many research-like tasks:

- locate the controlling clause in a standard,
- inspect prior assumptions,
- compare guidance across documents,
- justify why one interpretation is safer than another.

A system that organizes those steps can save time, but it can also mislead if it hides uncertainty or provenance.

Human-in-the-Loop (HITL)

Definition

A HITL system is one in which human review, intervention, or approval remains part of the operational workflow rather than being an afterthought.

In engineering, HITL is often a design requirement because the consequences of error can be physical, legal, financial, or ethical.

Where Humans Sit in the Loop

Depending on the task, the human may:

- specify goals and constraints,
- approve access to tools,
- review intermediate evidence,
- validate the final result,
- audit the workflow after completion.

Different placements imply different risk tolerances.

A Human-in-the-Loop Control Pattern

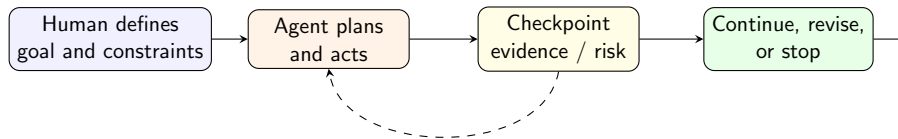


Figure: A HITL control pattern for agentic systems.

HITL as a Design Choice

If the human only sees the final polished answer, the loop may be too weak to matter.

Good HITL design requires exposing:

- intermediate evidence,
- constraints,
- uncertainty,
- provenance,
- decision points.

The point is operational judgment, not ceremonial approval.

Why Multi-Step Systems Fail Differently

Agentic systems inherit ordinary LLM weaknesses, but they also amplify them through multi-step behavior.

Key failure modes include:

- compounding error,
- tool misuse,
- poor escalation behavior,
- goal drift.

Compounding Error and Tool Misuse

A wrong retrieval, wrong unit, or wrong file choice can contaminate later steps.

Tool misuse adds more risk:

- wrong tool selected,
- wrong parameters passed,
- wrong output interpreted as valid.

Multi-step systems can therefore create longer chains of plausible but unsound behavior.

Escalation Failure and Goal Drift

A system may:

- escalate too often and become useless,
- escalate too rarely and become reckless,
- drift toward local completion instead of the real engineering goal.

Important Caution

Multi-step capability does not automatically create deeper understanding. It can create longer chains of confident error.

From Capability to Governance

As systems become more agentic, the question shifts from capability to governance.

Key questions become:

- who approved the actions,
- what evidence was used,
- what logs exist,
- who is responsible when the system fails,
- what should never happen without human review.

Auditability and Traceability

Engineering culture already values documentation and checkability.

An engineering agent should leave behind:

- its plan,
- actions,
- tool outputs,
- retrieved evidence,
- unresolved uncertainties.

That is how humans reconstruct whether a conclusion deserves trust.

Responsibility Cannot Be Delegated

Accountability for an engineering decision cannot simply be assigned to the model.

Governance Principle

As systems gain longer horizons, tool access, and partial autonomy, the burden of auditability and explicit responsibility increases rather than decreases.

Representative Systems in Practice

Before concluding, it helps to name systems that make agentic patterns concrete.

Three useful categories are:

- web-navigation and research agents,
- software-engineering agents,
- frameworks and cognitive architectures.

Web-Navigation and Research Agents

Examples include:

- **WebGPT** — browser-based search and cited answers,
- automated research assistants — compare sources, track gaps, synthesize,
- verification-driven orchestration frameworks such as VMAO.

Source: OpenAI: WebGPT — Zhang et al. (2026): VMAO

Software Engineering Agents

Agentic behavior is also visible in systems such as:

- GPT Engineer,
- MetaGPT,
- newer coding agents that inspect repositories, propose edits, run checks, and continue over many steps.

Software engineering is pedagogically useful because tool calls, retries, and verification are often visible in the workflow.

Frameworks and Architectures

At the framework level, students should recognize examples such as:

- LangChain-style agents,
- Toolformer-style tool-using language models,
- Soar-like architecture-level theories of intelligent behavior.

Practical Takeaway

Agentic AI is not one product category. It is a family of workflow designs that combine planning, tools, state, and review in different proportions.

Design Principles for Engineering Agentic Systems

A practical summary:

- ① use bounded autonomy,
- ② preserve evidence,
- ③ escalate intentionally,
- ④ design for recovery,
- ⑤ log the process,
- ⑥ keep the human decision where the risk is.

Putting the Pieces Together

The course progression now looks like this:

- ① Topic 15: language, retrieval, and evidence,
- ② Topic 16: usable systems built from prompts, retrieval, and tools,
- ③ Topic 17: agentic workflows that plan, act, maintain state, recover, and escalate.

The lesson is not to rush toward full autonomy. It is to decide how much agency is appropriate for which task.

Reaching Less-Charted Waters

At this point in the course, both students and instructor are reaching less-charted waters.

AI seems to be advancing in at least two directions:

- stronger foundation models, with new attention, memory, and possibly world-model-like paradigms,
- broader vertical applications, where many unmet needs may be accelerated by HITL workflows.

What Has Not Changed

Agentic systems can accelerate classical workflows, but many problems remain unresolved.

Realistic Interpretation

AI, at least before speculative ASI, primarily amplifies human intentions, institutions, and incentives rather than creating new purposes of its own.

This is not unusual; many revolutionary technologies have changed human activity through both leverage and disruption.

From Topic 17 to Topic 18

Topic 18 will take a broader, more speculative but still fact-based view of the future discussion around AI:

- neural-symbolic systems beyond current agentic workflows,
- world models,
- ASI,
- embedded intelligence.

We will ask whether we even understand how to create genuinely embodied intelligence before speaking confidently about AI that broadly surpasses humans.

Final Message

Personal Engineering Slogan

Regardless of the paradigm, engineers stay above AI. We design it, constrain it, audit it, and remain responsible for how it enters the world.